Transport Management System (TMS) — Software Specification Document

1. Project Overview

Attribute	Details
Project Name	Web-Based Transport Management System (TMS)
Project Type	Capstone Project / Continuous Assessment
Team Size	3 Members (2 Developers + 1 Product Manager)
Technology Stack	PHP, MySQL, HTML, CSS, JavaScript, Apache (LAMP Stack)
Development Server	LAMP Stack (Local Development)
Version Control	Git & GitHub
Deployment	Vercel (for PM testing/demo)

---

2. Project Objectives

Primary Goal

Develop a web-based system to manage fleet operations including vehicle tracking, driver assignment, trip scheduling, and maintenance logging.

Specific Objectives

- Provide a secure login system for administrators.
- Enable CRUD operations for vehicle records.
- Manage driver profiles and availability.
- Schedule and assign trips to vehicles and drivers.
- Track trip status (pending, in-transit, completed, cancelled).
- Log vehicle maintenance activities.
- Generate basic reports on fleet usage.

---

3. Functional Requirements

Module 1: Authentication & User Management

Requirement	Priority
Secure login with username/password	**High**
Password hashing for security	**High**
Session management (login/logout)	**High**
User roles (admin, staff)	**Medium**

Module 2: Dashboard

Requirement	Priority
Display total vehicles, drivers, trips	**High**
Show trip statistics (today, pending, completed)	**Medium**
Quick navigation to key modules	**Medium**

Module 3: Vehicle Management

Requirement	Priority
Add new vehicle (name, license plate, model, capacity)	**High**
View all vehicles in a list	**High**
Edit vehicle details	**High**
Delete vehicle records	**High**
Track vehicle status (available, assigned, maintenance)	**High**

Module 4: Driver Management

Requirement	Priority
Add new driver (name, license number, phone, email)	**High**
View all drivers in a list	**High**
Edit driver details	**High**
Delete driver records	**High**
Track driver status (available, on-trip, unavailable)	**High**

Module 5: Trip Management

Requirement	Priority
Create trip request (origin, destination, date, purpose)	**High**
Assign vehicle and driver to trip	**High**
View all trips with status	**High**
Update trip status (pending → approved → in-transit → completed)	**High**
Cancel trip if needed	**Medium**
Generate unique trip code for each trip	**High**

Module 6: Maintenance Management

Requirement	Priority
Log maintenance activities (date, description, cost)	**Medium**
Schedule future maintenance	**Low**
View maintenance history per vehicle	**Medium**

Module 7: Reports

Requirement	Priority
Trip history report	**Medium**
Vehicle utilization report	**Low**
Trip count by date range	**Low**

---

4. Non-Functional Requirements

Requirement	Description
Performance	Page load time under 3 seconds.
Security	Prevent SQL Injection (prepared statements) and Cross-Site Scripting (XSS) attacks.
Usability	Intuitive interface with clear navigation and consistent layout.
Compatibility	Works on modern browsers (Chrome, Firefox, Edge, Safari).
Responsiveness	Mobile-friendly responsive layout (mobile-first approach).
Code Quality	Clean, commented, well-organized code.
Data Integrity	Foreign key constraints enforced in the MySQL database.

---

5. Technology Specifications

Backend Stack

- **Server-Side Language**: PHP 8.x
- **Database**: MySQL 8.x
- **Server**: Apache 2.4
- **Database Extension**: MySQLi (Object-Oriented)
- **Session Management**: Native PHP Sessions

Frontend Stack

- **Markup**: HTML5
- **Styling**: CSS3 (Custom styling, mobile-first design)
- **Scripting**: Vanilla JavaScript (no external libraries unless needed)
- **Design Approach**: Mobile-first responsive

Development Tools

- **IDE**: Visual Studio Code
- **DB Administration**: phpMyAdmin / command line
- **Version Control**: Git & GitHub
- **Hosting/Testing**: Vercel (frontend deployment for PM testing)

---

6. Database Schema

Entity-Relationship Diagram

```
erDiagram
```

```
    admins {

        int id PK "AI"

        varchar fullname "100"

        varchar email UK "100"

        varchar password "255"

        enum role "super_admin, manager, staff"

        timestamp created_at

    }



    vehicles {

        int id PK "AI"

        varchar vehicle_name "100"

        varchar license_plate UK "50"

        varchar model "50"
```

```
        int capacity

        enum status "available, assigned, under_maintenance, out_of_service"

        timestamp created_at

    }



    drivers {

        int id PK "AI"

        varchar fullname "100"

        varchar license_number UK "50"

        varchar phone "20"

        varchar email "100"

        text address

        enum status "available, on_trip, unavailable"

        timestamp created_at
```

```
    }


    trips {

        int id PK "AI"

        varchar trip_code "50"

        date trip_date

        varchar origin "255"

        varchar destination "255"

        text purpose

        int vehicle_id FK "vehicles(id)"

        int driver_id FK "drivers(id)"

        enum status "pending, approved, in_transit, completed, cancelled"

        int created_by FK "admins(id)"

        timestamp created_at
```

```
    }


    maintenance {
        int id PK "AI"
        int vehicle_id FK "vehicles(id)"
        date maintenance_date
        text description
        decimal cost "10,2"
        date next_due_date
        enum status "scheduled, in_progress, completed"
        timestamp created_at
    }


    admins ||--o{ trips : "creates"
```

```
    vehicles ||--o{ trips : "assigned_to"

    drivers ||--o{ trips : "assigned_to"

    vehicles ||--o{ maintenance : "undergoes"
```

Table Details

Table: `admins`

Column	Type	Null	Key	Default	Extra / Description
`id`	INT	NO	PRI	NULL	Auto-increment
`fullname`	VARCHAR(100)	NO		NULL	Admin full name
`email`	VARCHAR(100)	NO	UNI	NULL	Login email (unique)
`password`	VARCHAR(255)	NO		NULL	Hashed password
`role`	ENUM('super_admin', 'manager', 'staff')	NO		'staff'	User access role
`created_at`	TIMESTAMP	NO		CURRENT_TIMESTAMP	Record creation date

Table: `vehicles`

Column	Type	Null	Key	Default	Extra / Description
`id`	INT	NO	PRI	NULL	Auto-increment
`vehicle_name`	VARCHAR(100)	NO		NULL	Vehicle name / identifier
`license_plate`	VARCHAR(50)	NO	UNI	NULL	License plate (unique)
`model`	VARCHAR(50)	NO		NULL	Vehicle model
`capacity`	INT	NO		NULL	Passenger/cargo capacity

Column	Type	Null	Key	Default	Extra / Description
status	ENUM('available', 'assigned', 'under_maintenance', 'out_of_service')	NO		'available'	Status
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Record creation date

Table: drivers

Column	Type	Null	Key	Default	Extra / Description
id	INT	NO	PRI	NULL	Auto-increment
fullname	VARCHAR(100)	NO		NULL	Driver full name
license_number	VARCHAR(50)	NO	UNI	NULL	Driver license (unique)
phone	VARCHAR(20)	NO		NULL	Contact number
email	VARCHAR(100)	YES		NULL	Email address
address	TEXT	YES		NULL	Physical address
status	ENUM('available', 'on_trip', 'unavailable')	NO		'available'	Status
created_at	TIMESTAMP	NO		CURRENT_TIMESTAMP	Record creation date

Table: trips

Column	Type	Null	Key	Default	Extra / Description
id	INT	NO	PRI	NULL	Auto-increment
trip_code	VARCHAR(50)	NO	UNI	NULL	Unique trip identifier
trip_date	DATE	NO		NULL	Date of trip
origin	VARCHAR(255)	NO		NULL	Starting location
destination	VARCHAR(255)	NO		NULL	Ending location
purpose	TEXT	YES		NULL	Trip purpose
vehicle_id	INT	YES	MUL	NULL	Foreign Key referencing vehicles.id

Column	Type	Null	Key	Default	Extra / Description
`driver_id`	INT	YES	MUL	NULL	Foreign Key referencing `drivers.id`
`status`	ENUM('pending', 'approved', 'in_transit', 'completed', 'cancelled')	NO		'pending'	Status
`created_by`	INT	NO	MUL	NULL	Foreign Key referencing `admins.id`
`created_at`	TIMESTAMP	NO		CURRENT_TIMESTAMP	Record creation date

Table: `maintenance`

Column	Type	Null	Key	Default	Extra / Description
`id`	INT	NO	PRI	NULL	Auto-increment
`vehicle_id`	INT	NO	MUL	NULL	Foreign Key referencing `vehicles.id`
`maintenance_date`	DATE	NO		NULL	Date of maintenance
`description`	TEXT	NO		NULL	Maintenance details
`cost`	DECIMAL(10,2)	NO		0.00	Cost of maintenance
`next_due_date`	DATE	YES		NULL	Scheduled next maintenance
`status`	ENUM('scheduled', 'in_progress', 'completed')	NO		'scheduled'	Status
`created_at`	TIMESTAMP	NO		CURRENT_TIMESTAMP	Record creation date

---

7. File Structure Specification

The project directories and files will be organized as follows:

```
/var/www/html/tms/
│
├── index.php                    (Redirect to login or dashboard)
├── login.php                    (Login page)
├── logout.php                   (Logout handler)
├── dashboard.php                (Dashboard page)
│
├── vehicles.php                 (Vehicle list)
├── add_vehicle.php              (Add vehicle form)
├── edit_vehicle.php             (Edit vehicle form)
├── delete_vehicle.php           (Delete vehicle handler)
│
├── drivers.php                  (Driver list)
├── add_driver.php               (Add driver form)
├── edit_driver.php              (Edit driver form)
```

```
├── delete_driver.php          (Delete driver handler)
│
├── trips.php                  (Trip list)
├── add_trip.php               (Add trip form)
├── edit_trip.php              (Edit trip form)
├── delete_trip.php            (Delete/cancel trip handler)
│
├── reports.php                (Reports page)
├── maintenance.php            (Maintenance logs)
│
├── includes/
│   ├── config.php             (Database connection)
│   ├── header.php             (Shared HTML header)
│   ├── footer.php             (Shared HTML footer)
│   └── functions.php          (Reusable functions)
```

```
│
├── css/
│   └── style.css              (Main stylesheet)
│
├── js/
│   └── script.js              (JavaScript functions)
│
└── assets/
    └── images/                (Logos, icons, etc.)
```

---

8. Module Specifications (Detailed CRUD)

Vehicle Module (Developer 1 - Backend)

Operation	PHP File	Description
Create	`add_vehicle.php`	Form page to add a new vehicle to the database.
Read	`vehicles.php`	Main list page showing all vehicles and their status.
Update	`edit_vehicle.php`	Form page to modify existing vehicle records.

Operation	PHP File	Description
Delete	`delete_vehicle.php`	Background handler to remove a vehicle record (requires confirmation first).

Driver Module (Developer 1 - Backend)

Operation	PHP File	Description
Create	`add_driver.php`	Form page to register a new driver with contact info.
Read	`drivers.php`	List page displaying all driver profiles and availability status.
Update	`edit_driver.php`	Form page to update contact info, license number, or status.
Delete	`delete_driver.php`	Background handler to remove a driver record (requires confirmation first).

Trip Module (Developer 2 - Frontend/Trips)

Operation	PHP File	Description
Create	`add_trip.php`	Form page to schedule a trip request, auto-generating a unique code, and assigning an available vehicle and driver.
Read	`trips.php`	Main trips view with status tracking indicators.
Update	`edit_trip.php`	Form page/modal to update trip progress (e.g., pending → approved → in-transit → completed).
Delete	`delete_trip.php`	Background handler to cancel/delete a scheduled trip.

Reports Module (Developer 2 - Frontend/Trips)

Operation	PHP File	Description
View	`reports.php`	Consolidated dashboard panel with trip history and usage analytics.
Filter	`reports.php?from=...`	Date-range filters using GET parameters to refine reports.

---

9. Team Roles and Responsibilities

Product Manager (PM)

- **Project Planning**: Create and maintain the overall project timeline.
- **Communication**: Act as the primary contact with the course coordinator/professor.
- **Task Tracking**: Manage the Trello or GitHub Projects board.
- **Testing**: Perform system testing on features after dev branch integration.
- **Documentation**: Write the user manual and draft the technical report.
- **Presentation**: Create final presentation slides.
- **Bug Tracking**: Log, triage, and track bug tickets.

Developer 1 (Backend & Database)

- **Database**: Design schemas and create the database initialization SQL script.
- **Configuration**: Write connection files (`includes/config.php`).
- **Auth System**: Build secure login & logout handlers (`login.php`, `logout.php`).
- **Vehicle CRUD**: Build `vehicles.php`, `add_vehicle.php`, `edit_vehicle.php`, and `delete_vehicle.php`.
- **Driver CRUD**: Build `drivers.php`, `add_driver.php`, `edit_driver.php`, and `delete_driver.php`.
- **Dashboard Backend**: Write the analytics metrics SQL queries for `dashboard.php`.
- **Security**: Implement password hashing and page-level session authorization.

Developer 2 (Frontend & Trip Module)

- **UI/UX Design**: Design layouts, responsive grids, and typography in `css/style.css`.
- **Templates**: Implement base template layout architecture (`includes/header.php`, `includes/footer.php`).
- **Trip CRUD**: Build `trips.php`, `add_trip.php`, `edit_trip.php`, and `delete_trip.php`.
- **Assignment Logic**: Formulate logic checks to ensure only available drivers and vehicles can be assigned.
- **Reports**: Code the date-filtering visual reports page (`reports.php`).
- **Interactivity**: Write Javascript elements (`js/script.js`) for notifications, form checks, and modals.

- **Maintenance Logs**: Build the vehicle maintenance logging module (`maintenance.php`).

---

10. Project Phases and Timeline

Phase	Target Duration	Key Activities / Milestones
Phase 1: Setup	Week 1	Repo setup, database initialization, workspace folders, base structure template layout.
Phase 2: Authentication	Week 2	Administrator tables, login/logout handlers, password hashing, session checking middleware.
Phase 3: Core Modules	Weeks 3–4	Vehicle and Driver database tables & full CRUD implementation with status changes.
Phase 4: Trips & Reports	Weeks 5–6	Trip creation logic, auto-assignment validation, maintenance forms, reports filtering.
Phase 5: Integration	Week 7	Pull requests audit, merging feature branches, system testing, bug remediation.
Phase 6: Submission	Week 8	Technical documentation polish, presentation recording, slide finalization.

---

11. GitHub Branching Strategy

Active Branches

- `main`: Holds production-ready, thoroughly tested codebase.
- `feature/backend-core`: Developer 1's workspace for DB config, Auth, Vehicles, and Drivers.

- `feature/frontend-trips`: Developer 2's workspace for CSS UI design, Trips, Reports, and Maintenance.

Developer Workflow

1. Work locally on your respective branch.
2. Commit changes frequently with clear, conventional messages.
3. Push branches to the remote repository.
4. Create a Pull Request (PR) to `main` once features are ready and self-tested.
5. Perform peer-review checks before merging code into `main`.
6. Regularly pull the latest changes from `main` into your feature branches to avoid major merge conflicts.

---

12. Security Requirements

Area	Implementation Details
Password Storage	Store credentials using native PHP `password_hash()` and check with `password_verify()`.
SQL Injection Protection	Enforce the use of prepared statements using MySQLi (`mysqli_prepare()`, `mysqli_stmt_bind_param()`).
Session Security	Invoke `session_regenerate_id(true)` upon successful user authentication to prevent session fixation.
XSS Protection	Sanitize outputs in HTML using `htmlspecialchars()` wrapper functions.
Brute Force Protection	*(Optional)* Restrict consecutive invalid login attempts.
Access Control	Enforce session validation checks at the top of all administration pages.

---

13. Testing Checklist

Test ID	Scenario	Input	Expected Result
TC-01	Correct Login	Valid admin username & password	Successfully redirects user to `dashboard.php`
TC-02	Incorrect Login	Invalid username or wrong password	Shows red validation error banner, keeps user on `login.php`
TC-03	Create Vehicle	Valid vehicle registration form entries	Inserts record to DB and lists new vehicle on `vehicles.php`
TC-04	Update Vehicle	Valid changes to capacity/model	Updates DB record; modifications appear in list
TC-05	Delete Vehicle	Trigger delete option & confirm	Removes vehicle record from list and DB
TC-06	Create Trip Assignment	Assign available driver and vehicle	Auto-generates trip code, sets status as `pending`
TC-07	Trip State Progression	Update trip to `approved`, then `in_transit`	Changes trip status; driver/vehicle availability updates
TC-08	Dashboard Summary	Navigate to admin panel dashboard	Metric cards render correct statistics for all vehicles, drivers, and trips
TC-09	Logout Handler	Trigger logout navigation button	Destroys current session, redirects to `login.php`

---

14. Submission Requirements

- **Project ZIP**: Clean archive of the `tms/` project folder (excluding Git files and local settings).
- **Git Repository**: GitHub repository URL with full commit history.
- **Technical Documentation**: Detailed PDF explaining project structure, database design,

and key components.
- **Database Script**: SQL export file (`tms_db.sql`) to initialize database tables and seed test admin accounts.
- **Visual walkthrough**: Screenshots/mockups of key UI screens (Login, Dashboard, Vehicle CRUD, Trip Assignments).

---

15. Acceptance Criteria

1. **Secure Authentication**: Admin users must log in and log out securely. Unauthorized URLs must redirect to the login page.
2. **Vehicle Management**: Complete CRUD operations for vehicles must work correctly without database errors.
3. **Driver Management**: Complete CRUD operations for drivers must work correctly.
4. **Trip Management**: Trips must support assigning vehicles and drivers, generating unique codes, and progressing status states.
5. **Dashboard**: Metrics counters must display correct database counts.
6. **Reports**: Basic filters must load historical records matching date ranges.
7. **No High Vulnerabilities**: Code must be free of SQL injection and Cross-Site Scripting (XSS) issues.
8. **Responsive Layout**: The user interface must be clean, readable, and functional on desktop and mobile screens.

---

16. Deployment Strategy

Deployment Environments

- **Local Development (`http://localhost/tms/`)**: Where developers build, run, and unit test features.
- **Testing/PM Sandbox (`https://tms-project.vercel.app`)**: Staging site managed via GitHub integrations for the Product Manager to test user stories.

Deployment Workflow

```
graph TD

    A[Developers write code] --> B[Commit & Push to Feature Branch]

    B --> C[Create Pull Request to main]

    C --> D[Peer Review & Merge to main]

    D --> E[GitHub Trigger webhook]

    E --> F[Vercel automates deployment to Staging]

    F --> G[PM tests on staging Vercel URL]

    G -- "Bug Found" --> H[Log in Trello/Google Doc]

    H --> A

    G -- "Approved" --> I[Ready for submission]
```

---

17. Documentation Requirements

- **User Manual**: Step-by-step instructions on setting up, logging in, creating vehicles/drivers, scheduling trips, and pulling reports.
- **Technical Report**: Code architecture explanation, folder/file structure rationale, and

entity-relationship database design description.

- **Presentation Slides**: Highlight target audience, key problems solved, system demo screenshots, implementation challenges, and lessons learned.